*** Steps For Communication Between AI Engine To Backend And Vice Versa

---

ENVs used

AI_Engine_BASE_URL =https://localhost:9099

backend_Base_URL = https://localhost:6060

token_URL= http://localhost:9090/realms/reading-first/protocol/openid-connect/token

client_id = reading-first-ocr-engine

client_secret = <secret>

---

Termenologies

Backend means book apis that reading first UI and mobile will interact with owned by Backend Team [Rashad]

ai-engine means the service that makes the ocr and audio generation owned by AI team [Ahmed & Waleed]

---

1- Backend Wants to notify AI Engine with pdf to make text extraction process (#enqueue job)

- *Load/get {ai-engine-bearer-token} to be trusted to call AI engine
- make post request to {AI_Engine_BASE_URL}/api/jobs
  header authorization {ai-engine-bearer-token}
  body {
    jobId : "<GUID>",
    jobType: "PAGES_IMG_AND_TEXT_EXTRACTION",
    fileId: "<pdf file backend guid>"
    folderId: "<logical folder id>"
    pathPrefix : "<physical folder Id>"
    sourceDownloadUrl :  "<pdf s3 url>"
  }

return 202 accepted

---

What does the AI engine do after that?

It is a thing related to the AI team managing these jobs.

but expectation could be managed in different ways !!

A background task will take the enqueued jobs and start with them in parallel or sequential. It is a team cause depending on their resources, that's to make the text and image extraction by downloading the pdf first and then  start the pipeline which will result images and texts.

note: at some times download pdf via the link could be expired, so that
The ai engine should request a new url from our backend to get a fresh url for download.

Example of download pdf via given link in job enqueue body
- call GET <s3 url>
return the <file> with 200 OK

Example of get refreshed url after existing one is expired
- *Load/get {backend-bearer-token} to be trusted to call backend
- call GET {backend_Base_URL}/api/files/{<file backend guid>}/get-download-url
  header authorization {backend-bearer-token}
-return 200 OK

```
{
   "url": "<s3 url>"
}
```

---

2- AI engine read the pdf and make text extraction "per page" and wants to send text and images to Backend
note: this file will be only the free text no highlighting no timing needed at this step
- *call upload cycle per (image) then (text) so one page should result one image file and one text file
-And after both file success:
-*Load/get {backend-bearer-token} to be trusted to call backend
   header authorization {backend-bearer-token}
   Call PATCH {backend_Base_URL}/api/jobs/{<jobId GUID>}/<pageNumber>/completed

```
   {
   "artifacts": [
      { "fileId": "<Image file id returns from init upload>", "artifactType": "Image" },
      { "fileId": "<Text file id returns from init upload>", "artifactType": "Text" }
   ]
   }
```

   return 200 OK.

incase of error in extracting file image or text you want to tell backend you failed and here are the reason
-*Load/get {backend-bearer-token} to be trusted to call backend
   header authorization {backend-bearer-token}
   Call PATCH {backend_Base_URL}/api/jobs/{<jobId GUID>}/<pageNumber>/failed

```
   body{
      reason : "..."
   }
```

   return 200 OK.

But before this step number 2, backend wants to know how many pages in pdf
so ai engine should give the number of pages in the pdf before start text and image extraction
-*Load/get {backend-bearer-token} to be trusted to call backend
header authorization {backend-bearer-token}
Call PATCH {backend_Base_URL}/api/jobs/{<jobId GUID>}/metadata
body{
pagesCount : 30 // for example
}
return 200 OK.

so the backend will consider that you finish your process after you completed the 30 pages whether they fail or succeed, if 29, no, the backend will wait for number 30 page !!

---

3- Backend now needs to call an AI engine to ask it to make the audio extraction.
- *Load/get {ai-engine-bearer-token} to be trusted to call AI engine
- make post request to {AI_Engine_BASE_URL}/api/jobs
header authorization {ai-engine-bearer-token}
body {
jobId : "<GUID>",
jobType: "PAGE_AUDIO_AND_TEXT_TIMING_GENERATION",
fileId: "<free text file backend guid>"
folderId: "<logical folder id>"
pathPrefix : "<physical folder Id>"
sourceDownloadUrl : "<free text file s3 url>",
pageNumber: <page number>
}

return 202 accepted
---
What will the AI engine do after that?
It is a thing related to the AI team to manage these jobs.
but expectation could be managed in a different way !!

A background task will take the enqueued jobs and start with them in parallel or sequential

to make the audio generation and json timing generation by downloading the free text first and then start the generation pipeline which result audio and timing json.

note: at some times download the free text via the link could be expired, so that
The ai engine should request a new url from our backend to get a fresh url for download.
as mentioned above.

---

4- AI engine read the free text file and make audio generation and want to send audio and json to Backend
note:
- *call upload cycle per (audio) then (json) so one page should result one audio file and one json file
-And after both file success:
-*Load/get {backend-bearer-token} to be trusted to call backend
header authorization {backend-bearer-token}
Call PATCH {backend_Base_URL}/api/jobs/{<jobId GUID>}/<Given pageNumber>/completed
{
"artifacts": [
{ "fileId": "<Audio file id returns from init upload>", "artifactType": "Audio" },
{ "fileId": "<json file id returns from init upload>", "artifactType": "JSON" }
]
}

return 200 OK.

incase of error in generating audio file or json you want to tell backend you failed and here are the reason
-*Load/get {backend-bearer-token} to be trusted to call backend
header authorization {backend-bearer-token}
Call PATCH {backend_Base_URL}/api/jobs/{<jobId GUID>}/<Given pageNumber>/failed
body{
reason : "..."
}
return 200 OK.

---

Global point :
at some time you want to mark whole job as failed i.e you cant parse or get the pdf

-*Load/get {backend-bearer-token} to be trusted to call backend
header authorization {backend-bearer-token}
Call PATCH {backend_Base_URL}/api/jobs/{<jobId GUID>}/failed
body{
reason : "..."
}
return 200 OK.

---

Helper Functions Details | 401 means unauthorized token or expired
1- Load/get {backend-bearer-token} to be trusted to call backend
-Check if you have a saved token in some where i.e app memory [recommendation]
-if you have it, check if it is still valid and not exceed (5 minutes minus 30 sec) then return it.
-if no then call bellow
Post <Token_URL> \
-header Content-Type application/x-www-form-urlencoded
data-urlencode grant_type=client_credentials
data-urlencode client_id=<client_id>
data-urlencode client_secret=<client_secret>
return 200 OK
{
"access_token": "<eyJhbGciO...>",
"expires_in": 300,
"token_type": "Bearer",
}

- then save the access_token in your storage i.e cache of your running app [recommendation]. then return it.

Note: no need to store such a token in db or other place but it is better not to call the token api before each request, just call it when you really need it [performance wise].
---

2- Call upload cycle per file

- *Load/get {backend-bearer-token} to be trusted to call backend
- Post {backend_Base_URL}/api/files/initialize-upload
header authorization {backend-bearer-token}
body
{"fileName":"<file name>","contentType":"<image/jpeg> or <application/json> or <audio/wav> or <text/plain>", "sizeInBytes":<file size>,"folderId":<folder id>,"pathPrefix":<pathPrefix>}

return 200 OK

```
{
   "fileId": "<file id guid in backend>",
   "upload": {
      "url": "<S3 Upload URL>",
      "requiredHeaders": {
         "Content-Type": "<content type>",
         "Content-Length": "<file size>"
      }
   }
}
```

Then make PUT request to the "<S3 Upload URL>" with same headers in <requiredHeaders> and content as <file>
it should return 200 OK.

Then confirm the file is uploaded with backend
- *Load/get {backend-bearer-token} to be trusted to call backend
then make Post {backend_Base_URL}/api/files/<file id guid in backend>/confirm-upload
 header authorization {backend-bearer-token}

return 200 OK.
---

- note: ai engine should make jobs idempotent to guarantee not enqueue it multiple time via checking if you already have this job GUID or not, if already enqueued return 202 accepted also as any fail means exponential backoff retrying from backend and after a while stalled/dead letter.

Once an AI engine api receives the job request it verifies it has not any Prev one in db or any data store via GUID given and if it has it then it should return success, if it has not it should save it in db or any store this job to be work on.